

Architecture Before Technology

Decide the structure first. The technology questions answer themselves.

Matthias Falland

Microsoft Data Platform MVP, 20+ Fabric implementations

The first question in the room

"Lakehouse or Warehouse?"

Minute five of the first project meeting.

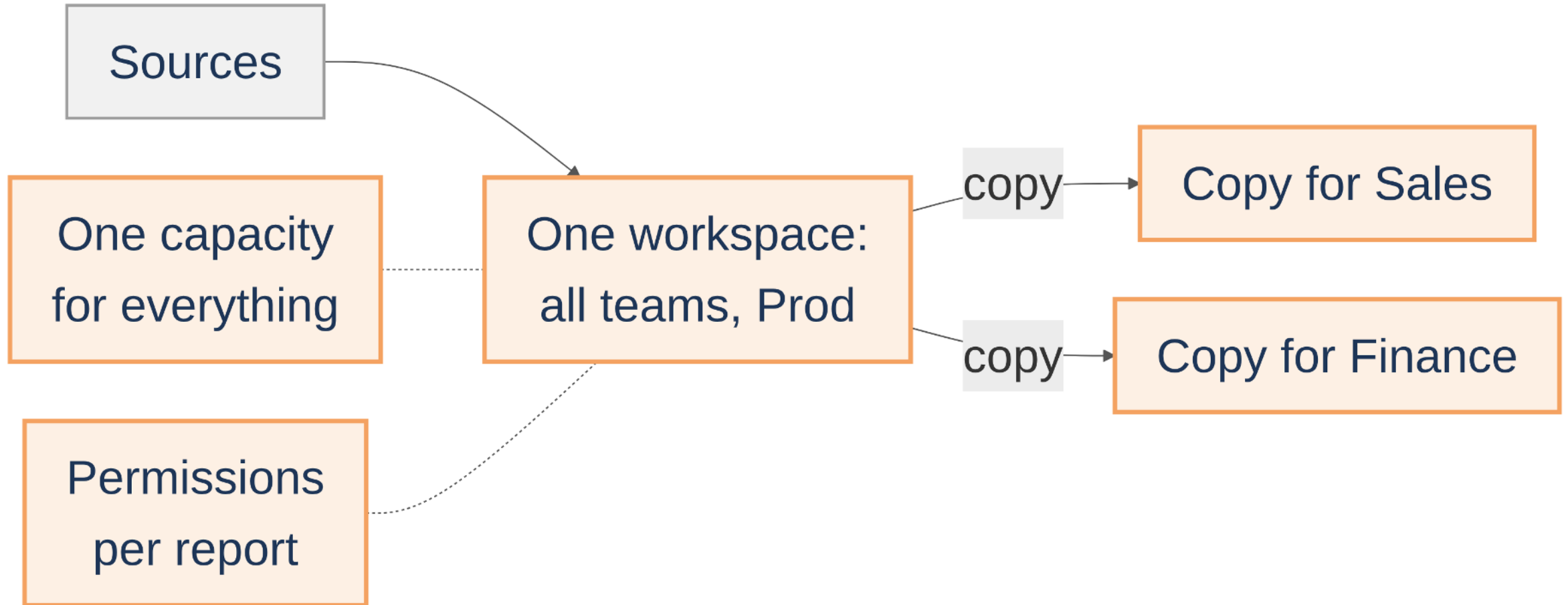
The right question, too early

It's the right question.

Asked too early.

Today: structure first, technology last.

Everything in one place



Five decisions that feel right

- One workspace for everything
- Copy the data to share it
- Everyone on one capacity
- Build directly in Prod
- Permissions report by report

Every one of them was reasonable

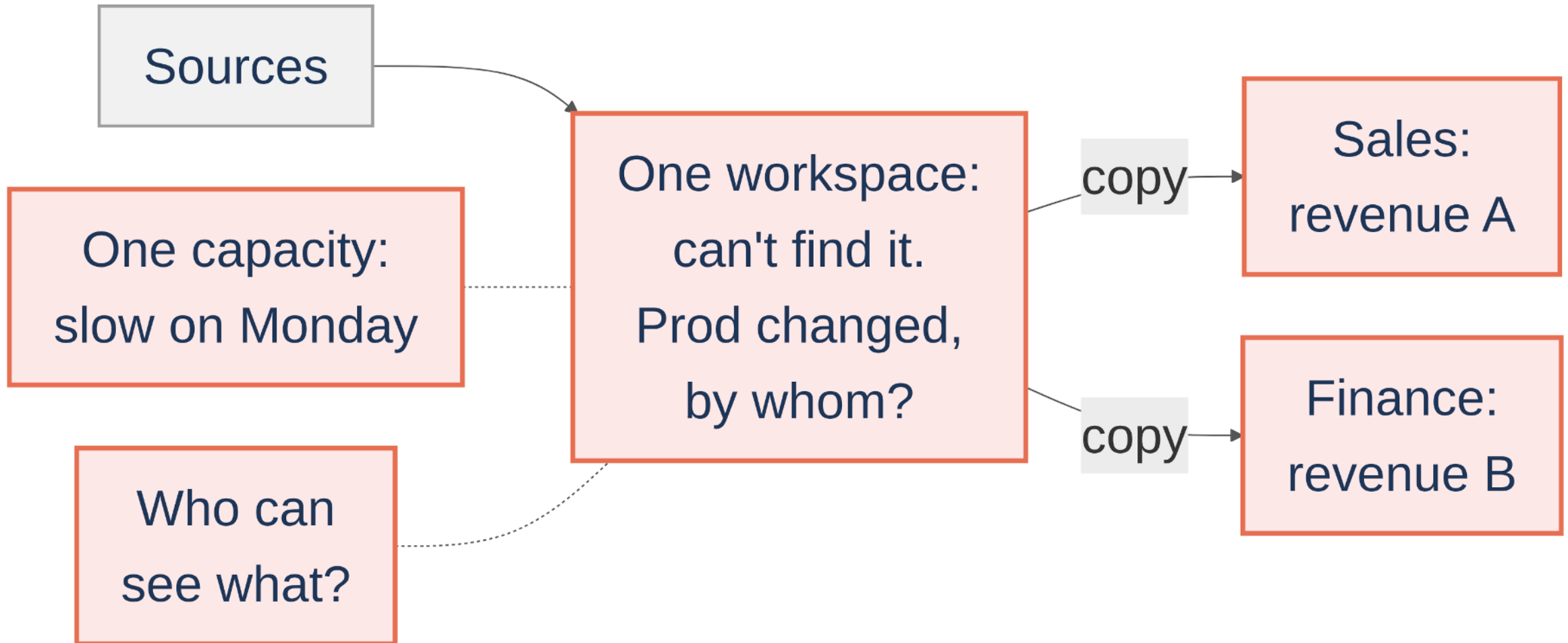
Every decision was reasonable.

None was about Lakehouse or Warehouse.

Now fast forward six months.

Month 6

The same picture, six months later



Five sentences you hear in month six

- "Where is the current version?"
- "Which revenue number is right?"
- "Why is it slow on Monday?"
- "Who changed Prod?"
- "Who can see this report?"

None of this is a technology problem

None of this is a technology problem.

Swapping Lakehouse for Warehouse fixes none.

Decide the structure first.

Workspaces by domain and stage

Finance-Prod-Bronze

Finance-Prod-Silver

Finance-Prod-Gold

Finance in Dev and Test: the same three layers

Sales-Dev-Reports

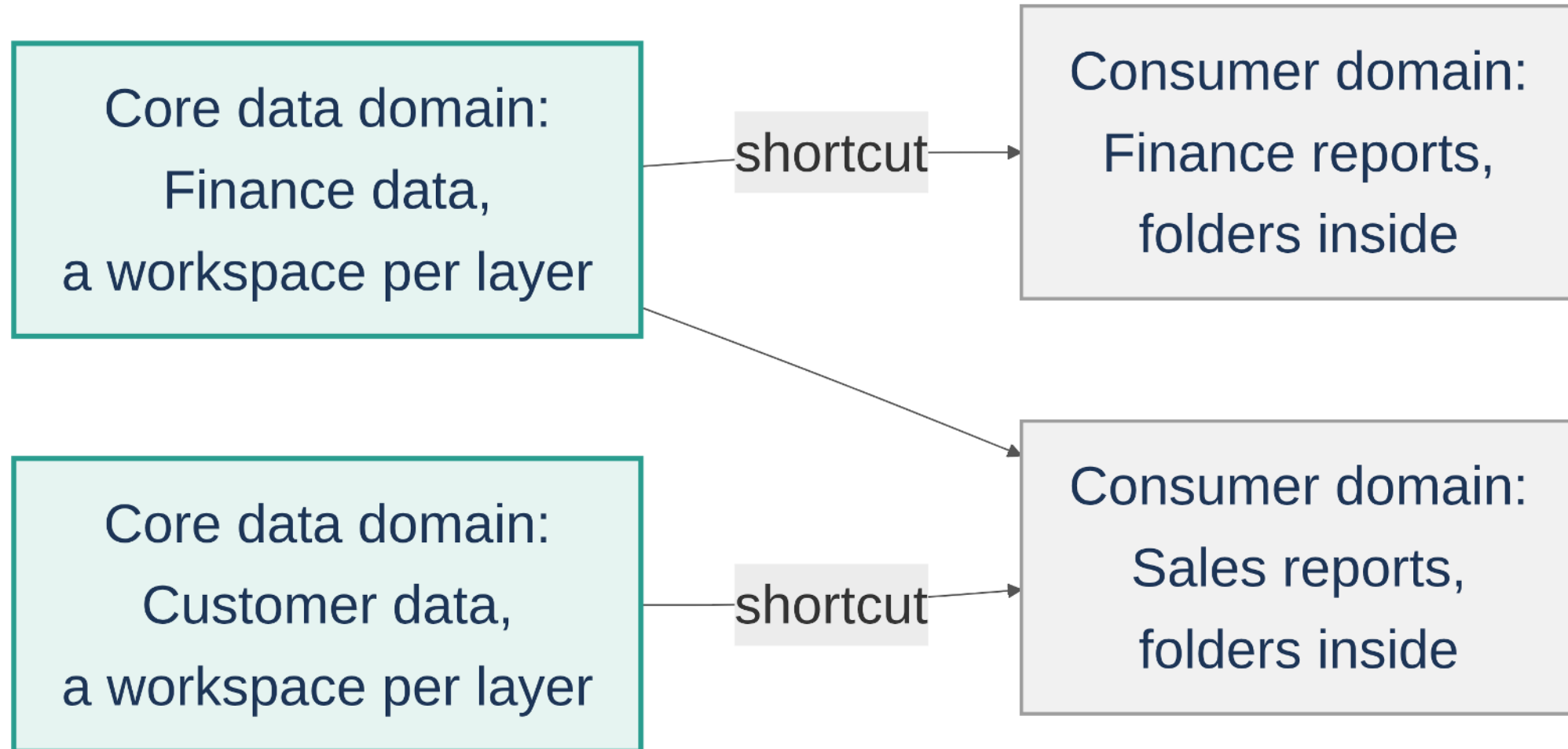
Sales-Test-Reports

Sales-Prod-Reports

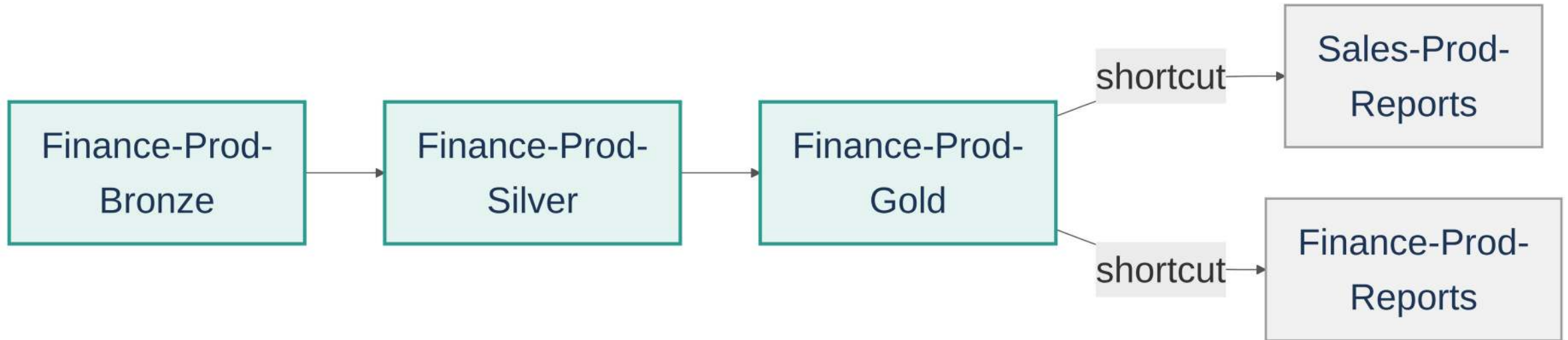
Names that tell you where you are

- {Domain}-{Env}-{Purpose}
- Finance-Prod-Gold
- Sales-Dev-Reports
- Sales-Prod-Reports
- Inside: folders per team

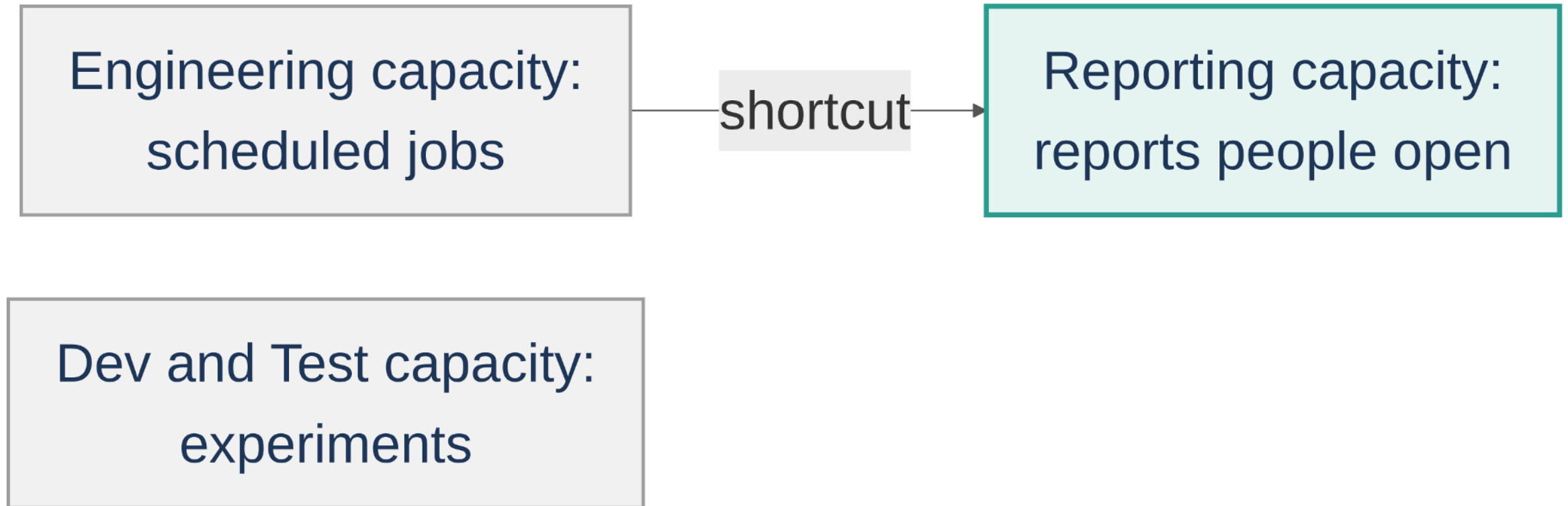
Core data domains and consumer domains



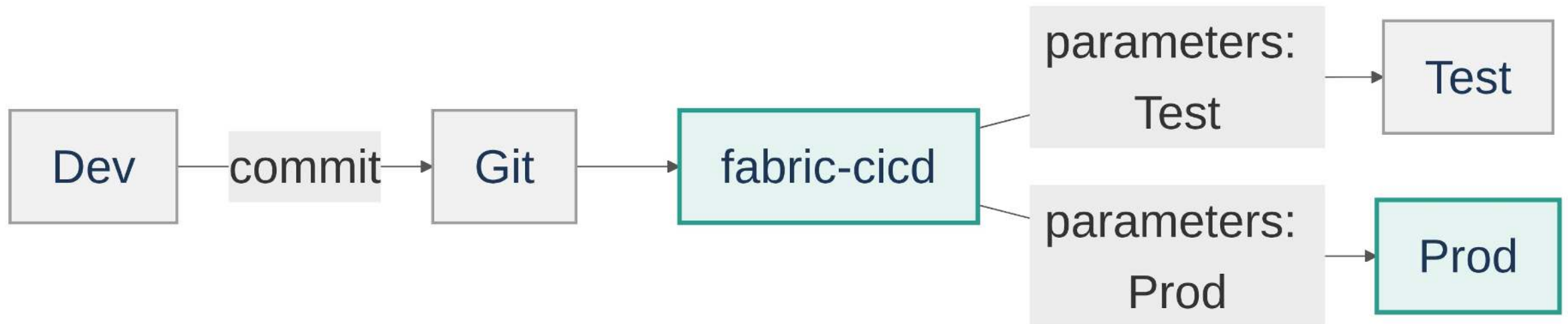
One copy: Bronze, Silver, Gold



Separate capacity by workload



Dev, Test and Prod: Git and fabric-cicd



Why I deploy with fabric-cicd

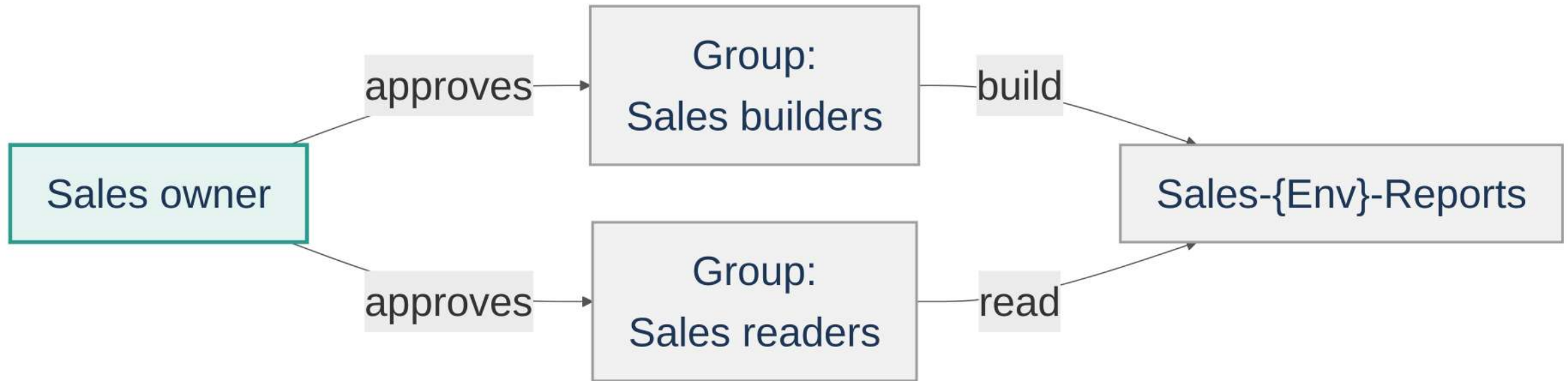
Every stage needs

- Its own connections
- Scheduled jobs
- Refresh after deploy

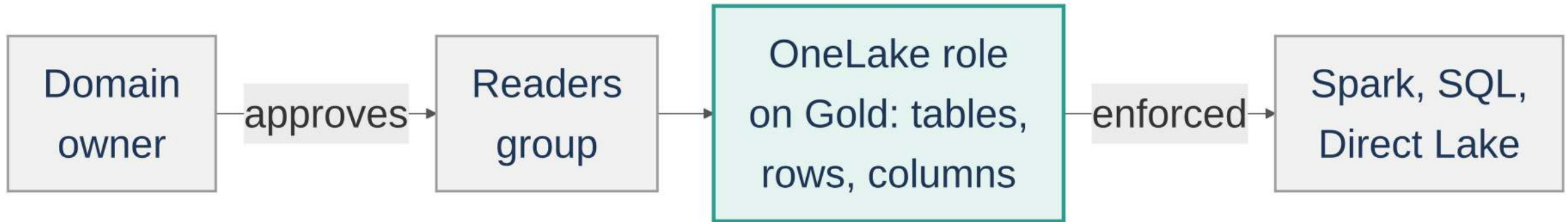
In the release

- Values per stage
- Schedules in Git
- Post-deploy refresh

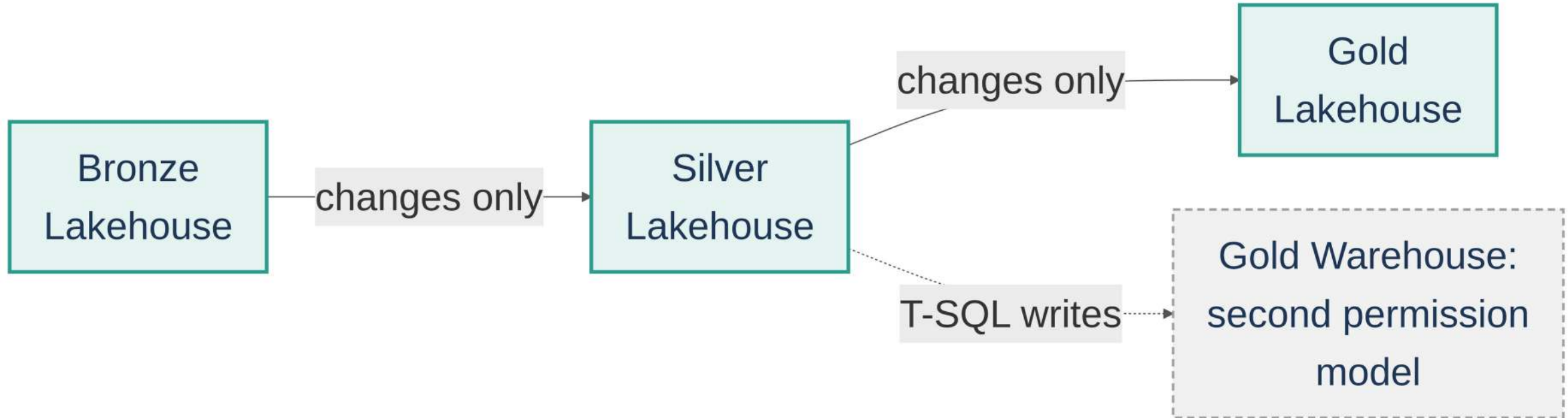
One owner per domain



Permissions once, at the data



Now it's easy: Lakehouse all the way



Keep the why

Write an ADR per decision

- ADR: decision, why, alternatives
- Date, owner, status
- Superseded, never edited
- ADRs in Git, versioned
- ADR-0001 Workspace cut
- ADR-0002 LH → LH → LH

Your architecture checklist

Decide first	Answer
1 Where does it live?	Workspace per domain and stage
2 How do others read it?	Shortcut to Gold, never a copy
3 What competes for compute?	Capacity per workload
4 How does it reach Prod?	Git, then fabric-cicd to Prod
5 Who says yes to access?	Owner per domain, roles in OneLake

Everything from today

Everything in one repo

Slides, handout, ADRs, links

github.com/TheTrustedAdvisor

[/fabcon-europe-2026](#)



Decide the structure first

Then the technology questions answer themselves.

Your month-six sentence:

[linkedin.com/in/matthias-falland](https://www.linkedin.com/in/matthias-falland)

QR: slides, handout, ADRs

